

UNIwersytet Ekonomiczny we Wrocławiu
Kierunek Informatyka w Biznesie

**Projekt i implementacja skalowalnego systemu orkiestracji
procesów biznesowych opartego na architekturze
sterowanej zdarzeniami**

Autor: Hubert Hącel
Promotor: Dr hab. inż. Marcin Hernes, prof.

Wrocław 2026

Spis treści

Wstęp	4
Streszczenie	5
Abstract	6
1. Zarządzanie procesami biznesowymi i automatyzacja procesów	7
1.1. Definicja i znaczenie zarządzania procesami biznesowymi	7
1.2. Automatyzacja, orkiestracja i wykonywanie workflow	7
1.3. Architektury stosowane w automatyzacji procesów	8
1.3.1. Systemy zarządzania przepływem pracy i BPMS	9
1.3.2. Integracja usług w architekturze SOA i orkiestracja BPEL	9
1.3.3. Rozproszona architektura sterowana zdarzeniami i mikrouslugi	9
1.4. Wyzwania automatyzacji procesów w systemach rozproszonych	10
1.4.1. Skalowalność i wydajność	10
1.4.2. Niezawodność komunikacji	11
1.4.3. Spójność stanu procesu	11
1.4.4. Obsługa błędów, timeoutów i ponowień	11
1.4.5. Przekazywanie danych między zadaniami	12
1.4.6. Obserwowalność i diagnostyka	12
1.4.7. Bezpieczeństwo automatyzowanych procesów	12
2. Analiza istniejących rozwiązań i technologii	13
2.1. Klasy systemów automatyzacji	13
2.1.1. Platformy BPMS	13
2.1.2. Systemy zarządzania przepływem pracy	14
2.1.3. Platformy orkiestracji workflow	14
2.1.4. Platformy automatyzacji zadań i procesów	14
2.2. Przegląd istniejących rozwiązań	15
2.2.1. Temporal	15
2.2.2. Apache Airflow	15
2.2.3. Camunda	16
2.2.4. Prefect	16
2.2.5. Rozwiązania chmurowe wspierające orkiestrację workflowów	16
2.3. Porównanie istniejących rozwiązań	17
2.4. Uzasadnienie wyboru podejścia projektowego	19
3. Architektura systemu rozproszonego i technologie wykorzystane w projekcie ..	20
3.1. Architektura sterowana zdarzeniami	20
3.2. Platforma komunikacji zdarzeniowej	20

3.3.	Konteneryzacja i Kubernetes	21
3.4.	Podsumowanie technologii wykorzystanych w projekcie.....	22
4.	Analiza i projekt systemu.....	23
4.1.	Cel i zakres systemu	23
4.2.	Wymagania funkcjonalne i нефункционалне.....	23
4.3.	Przypadki użycia	24
4.4.	Architektura systemu.....	26
4.4.1.	Komponenty i granice systemu	26
Bibliografia.....		29

Wstęp

W ostatniej dekadzie dynamiczny rozwój technologii cyfrowych oraz transformacja modeli operacyjnych przedsiębiorstw znacząco zmieniły podejście do zarządzania procesami biznesowymi. W celu zwiększenia efektywności operacyjnej i redukcji kosztów organizacje coraz częściej wdrażają zaawansowane rozwiązania informatyczne umożliwiające automatyzację złożonych łańcuchów czynności. Praca ta koncentruje się na projektowaniu i implementacji skalowalnego systemu orkiestracji procesów biznesowych opartego na architekturze sterowanej zdarzeniami.

Celem pracy jest zaprojektowanie, implementacja oraz przetestowanie autorskiego systemu orkiestracji procesów biznesowych spełniającego wymagania dotyczące skalowalności, odporności na awarie oraz wysokiej przepustowości. Zakres obejmuje przegląd istniejących rozwiązań, opracowanie architektury, implementację oraz przeprowadzenie testów wydajnościowych.

Streszczenie

Praca poświęcona jest projektowi i implementacji skalowalnego systemu orkiestracji procesów biznesowych opartego na architekturze sterowanej zdarzeniami. W ramach badań przeprowadzono przegląd istniejących podejść do automatyzacji procesów, ze szczególnym uwzględnieniem wzorców projektowych charakterystycznych dla systemów rozproszonych. Zaproponowano architekturę integrującą interfejs REST API, brokera komunikatów NATS z mechanizmem Queue Groups do automatycznego równoważenia obciążenia, bazę danych PostgreSQL oraz warstwę workerów skonteneryzowanych w środowisku Docker/Kubernetes. Testy wykazały odporność systemu na awarie, skuteczną dystrybucję zadań oraz minimalne narzuty czasowe podczas przetwarzania poleceń w odizolowanych procesach roboczych.

Słowa kluczowe: systemy rozproszone, architektura sterowana zdarzeniami, NATS, orkiestracja zadań, harmonogramowanie, Golang, mikrousługi, konteneryzacja.

Abstract

[TBD]

1. Zarządzanie procesami biznesowymi i automatyzacja procesów

W tym rozdziale przedstawione zostają kluczowe koncepcje zarządzania procesami biznesowymi, ewolucja podejść do ich automatyzacji. Omówiono również pojęcie workflow, orkiestracji, choreografii procesów, a także architektury stosowane w systemach automatyzacji. Szczególną uwagę poświęcono wyzwaniom związanym z automatyzacją procesów w środowiskach rozproszonych, w tym problemom skalowalności, niezawodności, spójności stanu, obsługi błędów i monitorowania.

1.1. Definicja i znaczenie zarządzania procesami biznesowymi

Zarządzanie procesami biznesowymi (ang. Business Process Management, BPM) stanowi podejście metodyczne, skupione na systematycznej identyfikacji, projektowaniu, wdrażaniu, monitorowaniu oraz optymalizacji procesów operacyjnych. Obejmuje ono cały cykl życia procesu od jego identyfikacji i modelowania, przez analizę oraz przeprojektowanie, po automatyzację i monitorowanie [1]. BPM nie ogranicza się przy tym do usprawniania pojedynczych czynności, lecz dotyczy całych łańcuchów działań, decyzji i zdarzeń, które ostatecznie tworzą wartość dla organizacji i jej klientów [1].

Znaczenie BPM we współczesnych organizacjach wzmacnia rozwój technologii automatyzacji. Manyika i współautorzy wskazują, że przy obecnie demonstrowanych technologiach około połowa aktywności za które pracownicy otrzymują wynagrodzenie, ma potencjał automatyzacji [2]. Nie oznacza to jednak, że przy obecnie demonstrowanych technologiach połowa miejsc pracy zostanie całkowicie zautomatyzowana. Autorzy rozróżniają automatyzację poszczególnych aktywności od pełnej automatyzacji całych zawodów i wskazują, że mniej niż 5% zawodów może zostać w pełni zautomatyzowanych przy wykorzystaniu obecnie dostępnych technologii [2].

Tempo i zakres wdrażania automatyzacji zależą między innymi od dalszego rozwoju technologicznego, kosztów technologii, dostępności kompetencji, relacji między kosztami pracy a kosztami automatyzacji, oczekiwanych korzyści wydajnościowych oraz akceptacji społecznej i regulacyjnej [2]. Z tego względu BPM pełni funkcje ramy organizacyjnej która pozwala identyfikować procesy wymagające usprawnienia, oceniać możliwości ich automatyzacji oraz monitorować skutki wprowadzonych zmian [1].

1.2. Automatyzacja, orkiestracja i wykonywanie workflow

Automatyzacja procesów biznesowych polega na wykorzystaniu technologii informatycznych do wykonywania działań, które wcześniej były realizowane ręcznie lub wymagały bezpośredniej ingerencji człowieka. Jej celem może być między innymi skrócenie czasu realizacji procesu, ograniczenie liczby błędów, zwiększenie powtarzalności działań oraz poprawa wykorzystania zasobów. Automatyzacja nie musi obejmować całego procesu. W praktyce często dotyczy wybranych zadań, takich jak walidacja danych, przesyłanie informacji między systemami, generowanie dokumentów lub uruchamianie operacji w zewnętrznych usługach [1].

Automatyzacja pojedynczych zadań nie rozwiązuje jednak problemu koordynacji całego procesu. W procesach wieloetapowych konieczne jest określenie kolejności wykonywania zadań, zależności między nimi, warunków przejścia do kolejnych etapów oraz sposobu reagowania na błędy. Z tego względu, automatyzacja procesów powinna być analizowana łącznie z mechanizmami zarządzania ich przebiegiem. BPM obejmuje bowiem nie tylko wykonywanie poszczególnych czynności, lecz także identyfikację, modelowanie, analizę, przeprojektowywanie i monitorowanie całego procesu [1].

Workflow można zdefiniować jako uporządkowany przebieg zadań, działań i decyzji prowadzących do osiągnięcia określonego rezultatu. Poszczególne zadania workflowu mogą być wykonywane sekwencyjnie lub równolegle, a rozpoczęcie danego zadania mogą być zależne od zakończenia wcześniejszych etapów, wartości danych wejściowych albo wystąpienia określonego zdarzenia. W systemie informatycznym workflow wymaga zatem nie tylko definicji zadań ale również mechanizmu odpowiedzialnego za interpretację zależności, uruchamianie kolejnych kroków i przechowywanie stanu wykonania [1].

Orkiestracja procesu oznacza sposób koordynowania przebiegu procesu w którym istnieje komponent odpowiedzialny za zarządzanie kolejnością i wykonaniem interakcji między uczestnikami systemu. Orkiestrator przechowuje lub interpretuje definicje procesu, uruchamia odpowiednie zadania, przekazuje dane między etapami oraz reaguje na wyniki ich wykonania.

Choreografia procesu opisuje współdziałanie wielu niezależnych niezależnych uczestników poprzez wymianę komunikatów i zdarzeń. W odróżnieniu do orkiestracji choreografia nie zakłada istnienia jednego centralnego komponentu który steruje całym procesem. Zamiast tego określa jakie komunikaty muszą być wymieniane między uczestnikami oraz jakie reakcje powinny nastąpić po wystąpieniu określonych zdarzeń. Busi i współautorzy wskazują, że choreografia opisuje system z perspektywy całościowej, podczas gdy orkiestracja koncentruje się na opisie i zarządzaniu pojedynczym uczestnikiem [3].

Orkiestracja i choreografia nie są podejściami wzajemnie wykluczającymi się. W jednym systemie orkiestrator może zarządzać przebiegiem głównego workflowu, natomiast poszczególne usługi mogą komunikować się ze sobą w sposób zdarzeniowy. W takim modelu orkiestrator podejmuje decyzje dotyczące przebiegu procesu, a zdarzenia informują pozostałe komponenty o zmianach stanu lub zakończeniu określonych operacji.

1.3. Architektury stosowane w automatyzacji procesów

Rozwój systemów automatyzacji procesów biznesowych przez lata budowany był w różnej architekturze, zaczynając od monolitycznych systemów zarządzania przepływem pracy BPMS, przez architektury SOA i BPEL aż po architekturę mikroserwisową i sterowaną zdarzeniami. Wszystkie mają wspólny cel, umożliwienia wykonywania procesów złożonych z wielu zadań.

1.3.1. Systemy zarządzania przepływem pracy i BPMS

Systemy zarządzania przepływem pracy (ang. Workflow Management Systems, WFMS) to rozwiązania które fundamentalnie. Ich podstawowym zadaniem jest określenie jakie, w jakiej kolejności i przez kogo lub przez co te czynności powinny być wykonane. Ich historycznym architektonicznym fundamentem jest centralizacja. W typowym WFMS przechowuje się model procesu, interpretuje go i bezpośrednio uruchamia zadania i zarządza całym stanem procesu. WFMS skupia się na sekwencji obsługi zadań z uwzględnieniem zadań ludzkich, czyli wykonywanych przez człowieka, w przypadku takich zadań system czeka na ręczną zmianę stanu poprzez zalogowanie się do systemu i odznaczenie zadania przez pracownika [4].

Szerszą koncepcją są Bussiness Process Management Systems (BPMS), które oprócz samego wykonywania workflowów, oferują możliwość modelowania procesów, ich analizy, a także optymalizacji. Architektonicznie są one monolityczne, gdyż logika biznesowa, interfejs graficzny oraz zarządzanie bazą danych są obsługiwane ze silną zależnością w jednym miejscu.

Słabą stroną tego rozwiązania jest przede wszystkim skalowanie oraz elastyczność, przy dużym obciążeniu skalowanie systemu jest trudne i kosztowne przez jego monolityczny charakter. Utrzymanie wszystkiego w jednym miejscu prowadzi również do uzależnienia od jednego dostawcy (ang. Vendor lock-in), czyli trudności do migracji do innych rozwiązań.

1.3.2. Integracja usług w architekturze SOA i orkiestracja BPEL

Kolejnym etapem w ewolucji architektur jest architektura zorientowana na usługi (ang. Service-Oriented Achitecture, SOA). W SOA monolityczne podejście WFMS, zostało rozbite na usługi które miały pełnić określone funkcje przez udostępnione interfejsy.

W SOA proces jest wykonywany przez wywoływanie wielu rozproszonych usług. Do zarządzania tą takimi systemami wykorzystuje się takie języki jak BPEL (ang. Business Process Execution Lanugage), w których orkiestrator zarządza przebiegiem interakcji. Orkiestrator komunikuje się z usługami poprzez zapytania SOAP/XML w sposób synchroniczny. Wywołuje on określoną usługę i czeka na odpowiedź, blokując tym samym swoje zasoby.

Chociaż logika procesu jest wydzielona do samych usług to architektura wciąż wykazuje silne powiązanie w warstwie komunikacyjnej. Synchroniczny charakter tego rozwiązania sprawia, że trudności z komunikacją się z jedną z usług może doprowadzić do kaskadowego zawieszenia całego procesu.

1.3.3. Rozproszona architektura sterowana zdarzeniami i mikrousługi

Architektura mikrousługowa podobnie jak w SOA dzieli system na niewielkie niezależne, wyspecjalizowane usługi. Mikrousługowa architektura pozwala na skalowanie każdej usługi niezależnie, w tej architekturze każda usługa może być niezależnie rozwijana, a upadek jednej nie zatrzyma pracy całego systemu.

Aby jednak rozproszone środowisko mogło realizować wspólny proces biznesowy konieczny będzie asynchroniczny sposób komunikacji [6]. Rozwiązaniem tego problemu może być architektura sterowana zdarzeniami (ang. Event-Driven Architecture, EDA). Nie korzysta ona z klasycznych zapytań lub wywołań, a zamiast tego używa asynchronicznej wymiany komunikatów poprzez brokery wiadomości.

W architekturze EDA komponenty są od siebie całkowicie odizolowane, usługa wysyłająca komunikat nie wie jeszcze kto go odbierze, wiadomość trafia do kolejki, z której kolejna usługa w momencie gotowości odbiera go i uruchamia swój proces.

1.4. Wyzwania automatyzacji procesów w systemach rozproszonych

Współpraca wielu komponentów w automatyzacji procesów biznesowych przy użyciu systemów rozproszonych doprowadza do powstania nowych wyzwań. Gdy komponenty działają na różnych maszynach, są rozwijane przez odrębne zespoły oraz korzystają z różnych technologii, a komunikacja między nimi odbywa się za pomocą sieci która może mieć opóźnienia i umożliwia całkowitą lub czasową utratę połączenia sposób wykonywania procesu stanowi klucz do minimalizowania ryzyka w większym stopniu, niż sama logika poszczególnych zadań.

W architekturach mikrousługowych, każda usługa odpowiada za ograniczony zakres funkcjonalny i może być wdrażana oraz skalowana niezależnie. Takie podejście ułatwia dopasowanie zasobów do obciążenia, ale również zwiększa liczbę interakcji między komponentami. Ta niezależność nie rozwiązuje jednak problemu współpracy przy realizacji procesów składających się z wielu etapów [6]. Wykonanie takiego procesu wymaga mechanizmu koordynującego działania usług i zapewniającego ciągłość przebiegu procesu.

1.4.1. Skalowalność i wydajność

Jednym z podstawowych wyzwań jest zapewnienie obsługi przy rosnącej liczbie procesów zleconych procesów. System powinien umożliwiać dodawanie kolejnych instancji komponentów, zamiast uzależniać wydajność od pojedynczego serwera lub procesu.

Skalowanie poziome nie ogranicza się jednak do zwiększania liczby komponentów odpowiedzialnych za wykonywanie zadań. Ograniczeniem może okazać się dowolny z komponentów, orkiestrator, broker komunikatów, baza danych albo sam serwer odpowiedzialny za komunikacje z użytkownikami.

Wydajność automatyzacji procesów powinna być oceniona za pomocą kilku kryteriów:

- Przepustowość
- Czas oczekiwania zadania na wykonanie
- Opóźnienie komunikacji
- Całkowity czas realizacji procesu

W przypadku procesów wieloetapowych jest również istotne określenie wpływu liczby zadań i zależności między nimi na czas wykonania.

1.4.2. Niezawodność komunikacji

Komunikacja sieciowa nie gwarantuje, że wiadomość zostanie dostarczona natychmiast, dokładnie raz i w oczekiwanej kolejności. Docelowa usługa może być niedostępna, a połączenie może zostać przerwane. System automatyzacji musi być gotowy na tego typu zjawiska i posiadać określony sposób działania w odpowiedzi na nie.

Należy również samo odebranie wiadomości przez serwis od udanego wykonania operacji w reakcji na nią. W systemach asynchronicznych trzeba uwzględnić możliwość ponownego dostarczenia tego samego komunikatu. Zjawisko to może wystąpić na przykład wtedy, gdy komponent odbierze komunikat, ale nie zdąży potwierdzić jego obsługi przed zerwaniem połączenia. Ponowne wykonanie operacji może doprowadzić do powielenia skutków ubocznych. Jednym ze sposobów ograniczenia tego ryzyka jest impotencja, czyli takie zaprojektowanie operacji, aby wielokrotne wykonanie tego samego zadania prowadziło do tego samego stanu końcowego.

1.4.3. Spójność stanu procesu

Gdy proces jest wieloetapowy, może być obsługiwany przez wiele usług na wielu osobnych maszynach. Konieczne jest więc przechowanie informacji o stanie całego procesu oraz jego zadań.

Stan procesu nie powinien być przechowywany wyłącznie w pamięci orkiestratora. W przypadku błędu orkiestratora, po ponownym uruchomieniu stany poszczególnych procesów byłyby utracone. Do przechowania stanu należy wykorzystać np. bazę danych. Tak by po restarcie, możliwe było wznowienie procesu i oczekujących zadań i ograniczenie ryzyka ponownego wykonania operacji, które zostały już zakończone.

1.4.4. Obsługa błędów, timeoutów i ponowień

Błąd jednego zadania w całym workflow, nie powinien zawsze kończyć się zatrzymaniem całego procesu. Zadanie zakończone błędem może być obsłużone w jeden z następujących sposobów:

- Ponowienie zadania
- Przekierowanie procesu do alternatywnej ścieżki
- Wykonanie operacji kompensującej
- Zakończenie procesu statusem błędu

Wybór sposobu powinien zależeć od procesu oraz zadania, w zależności od tego czy np. jego ponowne wykonanie jest bezpieczne.

Mechanizm ponowień (ang. *retry*), powinien być ograniczony liczbą prób lub maksymalnym czasem. Nieograniczone ponawianie może doprowadzić do zawieszenia systemu i utrudnić rozpoznanie awarii. Gdy usługa jest chwilowo niedostępna stosowanie opóźnień może być również dobrym pomysłem.

Każde zadanie powinno mieć określony limit czasu wykonania (ang. timeout). Timeout chroni system przed zablokowaniem przez operację, która nie zwraca odpowiedzi. W przypadku timeoutu zadanie powinno otrzymać adekwatny status.

1.4.5. Przekazywanie danych między zadaniami

Wiele procesów automatyzacji wymaga przekazania wyniku z jednego zadania na wejście kolejnego. Małe dane takie jak identyfikatory, statusy czy krótkie struktury JSON, mogą być przekazane bezpośrednio w komunikacie. Takie rozwiązanie jest proste, ale nie powinno być stosowane do dużych plików lub obszernych zbiorów danych.

W przypadku dużych wyników właściwsze jest zapisanie danych w zewnętrznym magazynie i przekazanie dalej wyłącznie identyfikatora artefaktu. Następny etap procesu może pobrać dane na podstawie tego identyfikatora. Ogranicza to rozmiar komunikatów ale wymaga określenia sposobu zarządzania cyklem życia danych, kontroli dostępu i reakcji na niedostępność magazynu.

Niezbędne okazać się może być również określanie „kontraktów” na dane między zadaniami. Taki kontrakt powinien pozwolić na wskazanie wymaganych pól, ich typy, oraz sposób interpretacji wartości. Podstawową formą sporządzenia takiego kontraktu jest np. JSON Schema.

1.4.6. Obserwowalność i diagnostyka

W systemie rozproszonym pojedynczy proces może przechodzić przez wiele usług, kolejek i jednostek wykonawczych. Analiza wyłącznie logów poszczególnych komponentów może nie pozwolić na odtworzenie pełnego przebiegu procesu. Potrzebne są informacje, które umożliwiają powiązanie komunikatów, zadań i operacji należących do tego samego wykonania.

Podstawowe elementy obserwowalności obejmują identyfikatory korelacyjne, statusy procesów, czasy rozpoczęcia i zakończenia zadań, liczbę ponowień oraz informacje o błędach. Warto również monitorować długość kolejek, liczbę aktywnych jednostek wykonawczych, przepustowość i zużycie zasobów.

Obserwowalność ma znaczenie nie tylko podczas diagnozowania awarii, lecz także przy ocenie wydajności i skalowalności. Bez odpowiednich metryk trudno ustalić, czy ograniczenie wynika z liczby jednostek wykonawczych, opóźnienia brokera, czasu działania zewnętrznej usługi czy niewystarczającej wydajności magazynu danych.

1.4.7. Bezpieczeństwo automatyzowanych procesów

Systemy automatyzacji mogą wykonywać operacje posiadające dostęp do danych organizacji, bezpieczeństwo więc, powinno obejmować zarówno komunikacje między komponentami, jak i kontrole uprawnień związanych z wykonywaniem zadań.

Istotna jest zasada najmniejszych uprawnień. Czyli ograniczenie możliwości komponentu do wykonywania określonych operacji niezwiązanych z zadaniem które ma wykonać. Ograniczenie uprawnień zmniejsza potencjalne skutki błędu lub wykorzystania podatności w jednym z zadań.

Należy także kontrolować możliwość tworzenia, uruchamiania, anulowania i obserwowania procesów. Dane uwierzytelniające powinny być przechowywane w bezpiecznym mechanizmie konfiguracji, a nie przekazywane jako jawne parametry komunikatów. W przypadku wykonywania poleceń systemowych konieczna jest dodatkowa walidacja danych wejściowych oraz izolacja środowiska wykonawczego.

2. Analiza istniejących rozwiązań i technologii

Rozdział ten opisuje jakie rozwiązania są już dostępne na rynku, z jakich technologii korzystają, oraz jakie korzyści z nich płyną. Zawiera porównanie tych rozwiązań biorąc pod uwagę kryteria opisane w poprzednim podpunkcie jak i wymagania biznesowe takie jak licencjonowanie i możliwości wdrożeniowe.

2.1. Klasy systemów automatyzacji

Na rynku znajdują się rozwiązania obejmujące na cel dostarczenie różnych funkcjonalności. Niektóre koncentrują się na modelowaniu i zarządzaniu procesami biznesowymi, inne zaś na wykonywaniu workflowów, koordynowaniu mikrousług lub automatyzacji pojedynczych zadań. Granice między poszczególnymi kategoriami nie są całkowicie jednoznaczne, ponieważ niektóre rozwiązania łączą funkcje kilku klas systemów.

Na potrzeby niniejszej pracy wydzielono cztery klasy rozwiązań:

- platformy BPMS
- systemy zarządzania przepływem pracy
- platformy orkiestracji workflow
- platformy automatyzacji zadań i procesów.

2.1.1. Platformy BPMS

Platformy BPMS należą do najszerzej kategorii rozwiązań. Ich zadaniem jest wspieranie w modelowaniu, wykonaniu, monitorowaniu oraz poprawianiu procesów biznesowych. Oprócz wykonywania procesu mogą zawierać narzędzia do modelowania, zarządzania regułami, integracji systemów, analizy przebiegu procesów oraz raportowania [4].

Klasa ta służy głównie organizacjom, które chcą zarządzać procesami w sposób ustandaryzowany i łączyć warstwę biznesową z warstwą techniczną. Jej silną stroną jest szeroki zakres funkcji i to, że wiele etapów cyklu życia procesu może być objęte przez pojedynczą platformę. Z drugiej strony odznaczają się one większą złożonością wdrożenia oraz koniecznością dopasowania platformy do organizacji.

2.1.2. Systemy zarządzania przepływem pracy

Systemy WFMS to węższa klasa rozwiązań, koncentrują się one na definiowaniu i wykonywaniu workflowów. Ich głównym zastosowaniem jest określanie kolejności czynności, zależności między zadaniami, oraz określanie zachowania w przypadku wyjątków. Zadania te mogą być realizowane przez aplikacje, usługi i komponenty wykonawcze, ale także przez ludzi.

W porównaniu z platformami BPMS, WFMS zwykle kładą mniejszy nacisk na modelowanie całej organizacji, zarządzaniem regułami biznesowymi i analitykę, natomiast są odpowiednie w sytuacjach w których przede wszystkim głównym celem jest kontrola przepływu pracy.

2.1.3. Platformy orkiestracji workflow

Platformy orkiestracji workflow skupiają się na koordynowaniu procesów wykonywanych przez wiele jednostek wykonawczych, czyli usług, aplikacji i komponentów. W porównaniu z klasycznymi systemami workflow, orkiestratory skupiają największą uwagę na problemy typowe dla środowisk rozproszonych:

- Trwałość wykonania
- Obsługa procesów długotrwałych
- Ponawianie zadań (ang. reprocessing)
- Timeouty
- Wznowienia po awarii
- Niezależne skalowanie komponentów

Rozwiązania z tej kategorii mogą być stosowane do wykonywania workflowów technicznych, procesów integracyjnych, zadań przetwarzania danych oraz procesów obejmujących wiele mikrousług. Przykładem podejścia łączącego orkiestrację z mikrousługami i komunikacją zdarzeniową jest platforma Beethoven opisana przez Barbozę i współautorów [6].

Głównym kryterium które odróżnia tę kategorię od ogólnych narzędzi automatyzacji jest nacisk na kontrolę stanu i niezawodność wykonywania procesu rozproszonego. Platformy orkiestracji często nie oferują pełnego modelowania procesów charakterystycznego dla BPMS, są bardziej techniczne i koncentrują się na koordynowaniu usług oraz zadań wykonanych przez infrastrukturę informatyczną.

2.1.4. Platformy automatyzacji zadań i procesów

Platformy automatyzacji zadań i procesów, służą do łączenia aplikacji, wyzwalania operacji oraz automatyzowania powtarzalnych czynności. Często wykorzystują one gotowe integracje, wywołania API, harmonogramy i wyzwalacze reagujące na zdarzenia zewnętrzne.

Do tej kategorii można zaliczyć różne typy narzędzi takie jak, Zapier i n8n, platformy integracyjne koncentrujące się na łączeniu aplikacji i wykonywaniu akcji po uruchomieniu określonego wyzwalacza. Zapier opisuje integracje za pomocą mechanizmów uwierzytelniania, wyzwalaczy i akcji [7], natomiast n8n służy do

automatyzacji workflowów i procesów biznesowych [8]. Z kolei narzędzia takie jak Apache Airflow skupiają się przede wszystkim na planowaniu i wykonywaniu przepływów zadań jak i zarządzaniu zależnościami między nimi [9]. Nie należy więc zakładać, że wszystkie platformy w tej kategorii mają taki sam zakres funkcjonalności.

Zaletą tych platform jest szybkie tworzenie automatyzacji i duża liczba gotowych integracji. Wady zależą od konkretnych rozwiązań i mogą dotyczyć sposobu przechowywania stanu, obsługi procesów długotrwałych, bezpieczeństwa czy obsługi wyjątków.

2.2. Przegląd istniejących rozwiązań

W tym rozdziale przedstawiono wybrane rozwiązania automatyzujące i orkiestrujące procesy biznesowe. Omówione platformy reprezentują różne podejścia do definiowania i wykonywania workflowów. Porównano je względem przeznaczenia, wcześniej wymienionych kryteriów oraz uwzględniono sposób wdrożenia i model licencjonowania. Kryteria te wpływają na możliwość zastosowania danego rozwiązania w określonym środowisku organizacyjnym. Celem nie jest wskazanie jednego rozwiązania jako najlepszego, a określić różnice między platformami oraz wskazać cechy które należy uwzględnić przy projektowaniu i ocenie systemu orkiestracji workflowów.

2.2.1. Temporal

Temporal jest platformą do wykonywania workflowów, których kroki mogą być realizowane przez niezależne komponenty wykonawcze. Szczególny nacisk położono w niej na trwałość wykonania, obsługę procesów długotrwałych, reprocessing, timeouts oraz możliwość wznowienia procesu po awarii. Operacje podatne na błędy są definiowane jako aktywności, które mogą być automatycznie ponawiane zgodnie z określoną polityką [10].

Istotną cechą Temporal jest rozdzielenie logiki workflowu od implementacji aktywności. Workflow określa przebieg procesu, natomiast aktywności reprezentują kolejne zadania z których Workflowy się składają. Temporal jest przydatny do orkiestracji procesów rozproszonych, ale wymaga zachowania deterministyczności kodu workflowu, oraz poprawnego projektowania operacji by uniknąć niepożądanych zachowań.

2.2.2. Apache Airflow

Apache Airflow jest platformą służącą do definiowania, planowania i wykonywania workflowów opartych na zadaniach. Workflow w Apache Airflow reprezentowany jest jako DAG, czyli graf zależności między zadaniami. DAG może zawierać informacje o harmonogramie, zadaniach, kolejności i regułach ich wykonywania oraz zachowaniu po zakończeniu całego procesu [9].

Airflow jest wykorzystywany do orkiestracji przepływów danych i zadań wsadowych. Platforma pozwala definiować zależności, wykonywać zadania równoległe, stosować rozgałęzienia oraz monitorować kolejne uruchomienia DAGów [9].

2.2.3. Camunda

Camunda jest platformą przeznaczoną do modelowania i wykonywania procesów za pomocą notacji BPMN. W tym podejściu przebieg procesu jest przedstawiany za pomocą graficznego modelu zawierającego zadania, zdarzenia, bramki logiczne i przepływy. Camunda zawiera narzędzia programistyczne, API i komponenty służące do wdrażania i monitorowania procesów [12].

Camunda kładzie duży nacisk na modelowanie procesów zrozumiałych dla analityków i zespołów technicznych organizacji. Dzięki BPMN może być wykorzystana do opisu procesów wykonywanych programowo jak i zadań wykonywanych przez ludzi.

2.2.4. Prefect

Prefect jest otwartym silnikiem orkiestracji, który pozwala przekształcać funkcje języka Python w workflowy. Workflowy mogą być tworzone bez odrębnego języka dziedzinowego lub rozbudowanych plików konfiguracyjnych, a następnie uruchamiane lokalnie, w kontenerach, w Kubernetesie, lub w środowisku chmurowym [13].

Platforma posiada funkcje śledzenia stanu, obsługi błędów, reprocessing, monitorowanie i możliwość wznowiania przerwanych zadań. Prefect wspiera uruchamianie workflowów według harmonogramu, w reakcji na wyzwalacz lub przez API [13]. Jest szczególnie przydatny w przepływach danych i automatyzacjach wymagających elastycznego modelu wykonania.

2.2.5. Rozwiązania chmurowe wspierające orkiestrację workflowów

Usługi chmurowe oferują funkcje orkiestracji bez konieczności utrzymywania infrastruktury. Przykładem jest Azure Durable Functions, w którym funkcja orkiestratora koordynuje wykonanie niezawodnego, długotrwałego workflowu. Orkiestracje mogą być definiowane w kodzie, procesowane synchronicznie lub asynchronicznie oraz automatycznie zapisywać postęp wykonania [14].

Zaletą usług chmurowych jest ograniczenie obowiązków związanych z instalacją, utrzymaniem i skalowaniem platformy. Użytkownik może skoncentrować się na definicji workflowu i logice aplikacji, podczas gdy dostawca odpowiada za udostępnienie środowiska wykonawczego.

2.3. Porównanie istniejących rozwiązań

Tabela 1 przedstawia porównanie wybranych platform według kryteriów wynikających z wyzwań automatyzacji procesów w systemach rozproszonych opisanych w podrozdziale 1.4. Uwzględniono także sposób wdrożenia i model licencjonowania, mają one wpływ na koszty utrzymania, zakres kontroli nad infrastrukturą oraz możliwość zastosowania platformy w określonym środowisku.

Porównywane rozwiązania różnią się przede wszystkim przeznaczeniem. Wszystkie platformy umożliwiają definiowanie zależności między zadaniami, ale różnią się sposobem reprezentowania procesu i zakresem obsługi stanów. Kryteria związane z systemami rozproszonymi pozwalają ocenić nie tylko dostępność funkcji, ale przede wszystkim niezawodność całego procesu.

Rozwiązanie	Główne zastosowanie	Definiowanie Workflow	Śledzenie stanu i niezawodność	Wdrożenie
Temporal	Orkiestracja procesów długotrwałych i rozproszonych.	Workflow i activities definiowane w kodzie.	Trwała historia wykonania, retry i timeouty; aktywności mogą zostać wykonane więcej niż raz, dlatego powinny być idempotentne.	Self-hosted lub usługa chmurowa, zależnie od wybranego modelu wdrożenia.
Apache Airflow	Orkiestracja przepływów zadań, szczególnie workflowów związanych z przetwarzaniem danych.	DAG definiujący zadania, harmonogram i zależności.	Obsługa zależności, ponowień, timeoutów i różnych reguł uruchamiania zadań.	Może być wdrażany samodzielnie lub korzystać z usług zarządzanych.
Camunda	Modelowanie i wykonywanie procesów biznesowych za pomocą BPMN.	Graficzny model procesu BPMN oraz komponenty wykonawcze platformy.	Obsługa procesów i integracji zależna od wykorzystanych komponentów platformy.	Camunda SaaS lub wdrożenie self-managed.
Prefect	Orkiestracja workflowów i produkcyjnych przepływów danych.	Workflowy definiowane w natywnym kodzie Pythona.	Śledzenie stanu, obsługa błędów, retry i możliwość wznowienia przerwanych wykonań.	Lokalnie, w kontenerach, Kubernetesie, self-hosted lub w chmurze.
Azure Durable Functions	Zarządzane chmurowo workflowy długotrwałe i orkiestracja funkcji.	Orkiestrator definiowany w kodzie, wywołujący funkcje synchronicznie i asynchronicznie.	Automatyczne zapisywanie postępu, historia wykonania i obsługa długotrwałych instancji	Usługa Azure, ograniczająca konieczność samodzielnego utrzymywania infrastruktury.

Tabela 1 - Porównanie istniejących rozwiązań orkiestracji procesów

2.4. Uzasadnienie wyboru podejścia projektowego

Przedstawione uprzednio rozwiązania posiadają szeroką funkcjonalność w zakresie orkiestracji i automatyzacji, są jednak przeznaczone do różnych zastosowań, w różny sposób definiują modele czy wykonywane workflowy. Zastosowanie gotowej platformy nie zawsze pozwala jednak na spełnienie wszystkich istotnych dla projektu wymagań, takich jak specyficzna obsługa stanu procesu, komunikacja między zadaniami, reagowanie na błędy czy zachowanie systemu po awarii.

Wiele organizacji dlatego decyduje się na opracowywanie własnego rozwiązania. W ramach tej pracy zdecydowano się na opracowanie własnego prototypowego rozwiązania, nie po to by stworzyć konkurencyjny produkt na rynku orkiestratorów, lecz by poznać wyzwania z którymi muszą się zmierzyć organizacje decydujące się na takie podejście i poznać w jaki sposób można zaimplementować mechanizmy niezbędne do poprawnego wykonywania workflowów.

Przyjęte podejście pozwoli również przeprowadzić testy działania systemu w różnych sytuacjach jak między innymi podczas wykonywania zadań w sekwencji, równolegle, zachowania w przypadku błędu, przekroczenia czasu wykonywania lub niedostępności komponentu.

Wybór podejścia projektowego wynika z potrzeby praktycznego zbadania problemu, a nie próby wypełnienia luki na rynku.

3. Architektura systemu rozproszonego i technologie wykorzystane w projekcie

W tym rozdziale przedstawiona została architektura oraz technologie wykorzystane do implementacji systemu orkiestracji workflowów. Skoncentrowano się na praktycznej realizacji mechanizmów przedstawionych w poprzednich rozdziałach. W szczególności na wykorzystaniu architektury sterowanej zdarzeniami, asynchronicznej komunikacji i wydzieleniu warstwy koordynującej wykonywane procesy i warstwy która te procesy wykonuje.

3.1. Architektura sterowana zdarzeniami

Architektura sterowana zdarzeniami zakłada komunikację między komponentami za pomocą zdarzeń i komunikatów, zamiast bezpośredniego wywoływania operacji przez jeden komponent. Komponenty mogą publikować informacje o zmianie stanu, polecenia lub zdarzenia, a inne komponenty reagują na nie zgodnie ze swoim przeznaczeniem [6].

W takim modelu należy rozróżnić zdarzenia, polecenia i komunikaty. Zdarzenie informuje o fakcie, który już wystąpił, na przykład o zakończeniu zadania. Polecenie jest żądaniem wykonania operacji, na przykład wykonaniem jakiegoś zadania. Natomiast komunikat to informacja przekazywana między komponentami i może też reprezentować zarówno polecenie jak i zdarzenie [6].

W projektowanym systemie, architektura sterowana zdarzeniami została wykorzystana przede wszystkim do komunikacji pomiędzy komponentem odpowiedzialnym za orkiestrację procesów, a komponentami odpowiedzialnymi za ich wykonywanie. Orkiestrator publikuje polecenia zadań do wykonania, a workerzy które wykonują te zadania pobierają polecenia z kolejki, wykonują polecenie i wyniki przekazują dalej na kolejkę. W tym układzie broker komunikatów pełni rolę pośrednika dzięki czemu komponenty nie muszą utrzymywać bezpośrednich połączeń między sobą [3].

3.2. Platforma komunikacji zdarzeniowej

Obecnie na rynku funkcjonuje wiele platform obsługujących komunikację zdarzeniową takie jak RabbitMQ, Kafka, Google Pub/Sub, Azure Service Bus. Do naszego prototypu jako platformę do komunikacji zdarzeń wykorzystano platformę NATS. Pełni ona funkcję pośrednika komunikacyjnego pośredniczącego w wymianie danych między usługami.

NATS wykorzystuje model publikowania i subskrypcji. Producent publikuje komunikat i adresuje go na dany temat (ang. subject), natomiast konsument go odbiera poprzez odpowiednią subskrypcję do tego tematu. Gwarantuje to niezależność pomiędzy komponentami [14].

NATS został wykorzystany do przekazywania poleceń wykonania zadań oraz wyników ich realizacji. Orkiestrator publikuje komunikat zawierający informacje potrzebne do uruchomienia zadania, natomiast worker odbiera to polecenie, wykonuje je i publikuje

komunikat z wynikiem. Dzięki temu, nasz orkiestrator będzie w stanie śledzić niezależne wykonywanie kolejnych zadań w workflow.

NATS umożliwia również tworzenie grup konsumentów, w których wiele instancji tej samej usługi może obsłużyć nadchodzący komunikat, pozwala to na rozłożenie obciążenia i w efekcie zapobiegnięcia oczekiwaniu na dostępność workerów [14].

W projekcie zastosowano również warstwę abstrakcji. Implementacja uniwersalnego interfejsu do obsługi naszego brokera komunikatów uniezależnia projekt od konkretnej implementacji jednego brokera, umożliwiając w przyszłości w razie potrzeby wymianę NATS na dowolnego innego brokera.

3.3. Konteneryzacja i Kubernetes

Docker umożliwi konteneryzację naszych serwisów. Podstawą działania Dockera są obrazy i kontenery. Obrazy (ang. images) to niezmiennie pakiety zawierające aplikacje i jej zależności oraz informacje potrzebne do jej uruchomienia. Obraz stanowi wzorzec na którego podstawie tworzone są kontenery, czyli uruchomione instancje obrazów w odizolowanych i powtarzalnych środowiskach wykonawczych [15].

Konteneryzacja pozwala uruchamiać poszczególne komponenty systemu jako niezależne usługi, w naszym projekcie jako osobne kontenery będą uruchamiane orkiestrator i osobno każdy z workerów oraz usługi pomocnicze.

Kubernetes służy do zarządzania skonteneryzowanymi aplikacjami. Pozwala na uruchamianie wielu replik usług, restartowanie ich w razie awarii oraz kontrolowanie dostępności komponentów [16]. Szczególne znaczenie ma możliwość poziomego skalowania workerów. Zwiększenie liczby instancji workerów pozwoli rozdzielać zadania między nimi.

3.4. Podsumowanie technologii wykorzystanych w projekcie

W projekcie wykorzystano zestaw technologii odpowiadających za spełnienie potrzeb poszczególnych elementów projektowanego systemu. Głównym językiem implementacji jest Go, w którym opracowano orkiestrator oraz komponenty workerów. NATS odpowiada za asynchroniczną komunikację pomiędzy usługami, a PostgreSQL za przechowywanie danych i stanów procesu. Natomiast Docker i Kubernetes za konteneryzację i zarządzanie środowiskiem uruchomieniowym.

Obszar systemu	Wykorzystane rozwiązanie	Rola w projekcie
Język implementacji	Go	Implementacja orkiestratora, workerów i innych usług systemu
Interfejs systemu	REST API	Przyjmowanie żądań oraz udostępnianie funkcji systemu
Orkiestracja	Autorski silnik workflow	Koordinowanie przebiegu workflowów i zależności między zadaniami
Komunikacja	NATS	Przekazywanie poleceń i wyników pomiędzy komponentami
Przechowywanie danych	PostgreSQL	Przechowywanie danych workflowów, zadań i ich stanów
Konteneryzacja	Docker	Budowanie obrazów i uruchamianie komponentów w kontenerach
Zarządzanie środowiskiem	Kubernetes	Uruchamianie, restartowanie i skalowanie usług

Tabela 2 – Wykorzystane technologie i ich rola w projekcie

4. Analiza i projekt systemu

W tym rozdziale przedstawiono analizę oraz projekt systemu, nazwanego roboczo TaskFlow. Jego zadaniem jest orkiestracja workflowów w środowisku rozproszonym. Na podstawie problemów i wyzwań omówionych we wcześniejszych rozdziałach określono cel systemu, jego zakres oraz wymagania funkcjonalne i нефункционалне, następnie przedstawiono przypadki użycia oraz zaprojektowaną architekturę systemu. Opis obejmuje poszczególne warstwy rozwiązania, model workflowów i zadań oraz zależności między zadaniami, sposób przechowywania danych i stany procesu.

4.1. Cel i zakres systemu

Celem projektu jest zaprojektowanie i implementacja prototypowego systemu TaskFlow przeznaczonego do orkiestracji workflowów w środowisku rozproszonym. System powinien umożliwiać koordynowanie wieloetapowych procesów, których poszczególne zadania mogą być wykonywane przez niezależne komponenty. Projekt pozwala na praktyczne zbadanie sposobu działania orkiestratorów oraz wykonywania workflowów w architekturze sterowanej zdarzeniami.

Zakres obejmuje opracowanie podstawowych elementów systemu odpowiedzialnych za definiowanie workflowów, koordynowanie ich przebiegu oraz wykonywanie składających się na nie zadań. System powinien umożliwiać skalowanie komponentów wykonawczych.

Projekt ma charakter prototypowy i nie ma na celu stworzenia kompletnej platformy, powinien umożliwiać zbadanie wybranych mechanizmów orkiestracji i ich działania w systemie rozproszonym.

4.2. Wymagania funkcjonalne i нефункционалне

W tym podpunkcie, zdefiniowane zostały wszystkie wymagania funkcjonalne oraz нефункционалне postawione przed projektem TaskFlow.

Wymagania Funkcjonalne
System powinien umożliwiać utworzenie workflowu składającego się z jednego lub większej liczby zadań.
System powinien umożliwiać określenie kolejności oraz zależności między zadaniami workflowu.
System powinien przekazywać zadania do workerów odpowiedzialnych za ich wykonanie.
System powinien odbierać i rejestrować wyniki wykonania zadań.
System powinien przechowywać informacje o stanie workflowu i poszczególnych zadań.
System powinien umożliwiać wykonywanie zadań sekwencyjnie oraz równoległe, jeżeli wynika to z ich zależności.
System powinien obsługiwać nieudane wykonania zadań, w tym ponowienie wykonania zgodnie z przyjętą konfiguracją.
System powinien ograniczać czas wykonywania zadania za pomocą mechanizmu timeoutu.
System powinien udostępniać informacje o stanie workflowu oraz wynikach wykonania zadań.
System powinien umożliwiać wykrywanie niedostępności lub zakończenia działania komponentów wykonawczych.

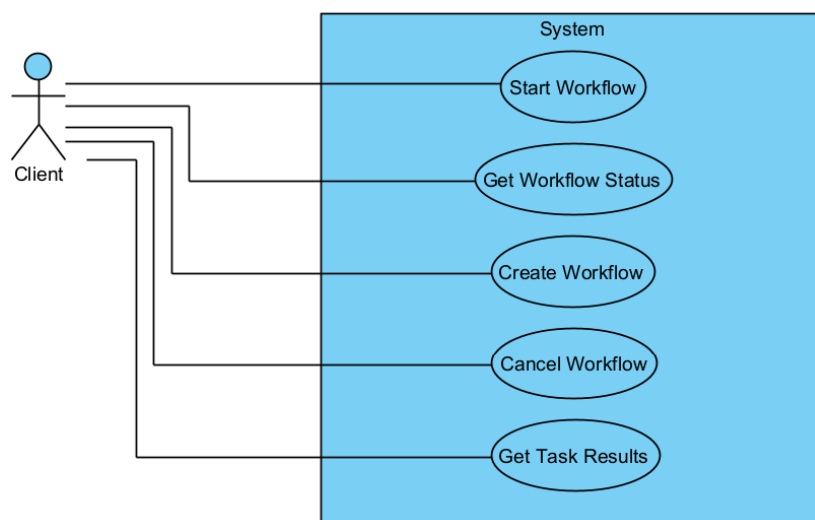
Tabela 3 – Wymagania Funkcjonalne

Wymagania Niefunkcjonalne
System powinien umożliwiać niezależne skalowanie komponentów odpowiedzialnych za wykonywanie zadań.
System powinien wykorzystywać asynchroniczną komunikację między komponentami.
System powinien zachowywać spójność informacji o stanie workflowu i jego zadań.
System powinien reagować na błędy komunikacji oraz nieprawidłowe zakończenie pracy komponentów.
System powinien umożliwiać ponowne uruchomienie komponentu bez konieczności ręcznego odtwarzania całego środowiska.
System powinien zapewniać możliwość monitorowania stanu usług oraz wykonywanych workflowów.
System powinien działać w powtarzalnym, skonteneryzowanym środowisku uruchomieniowym.
System powinien umożliwiać przeprowadzenie powtarzalnych testów funkcjonalnych, wydajnościowych i odpornościowych.

Tabela 4 – Wymagania Niefunkcjonalne

4.3. Przypadki użycia

Diagram przypadków użycia przedstawia funkcje systemu TaskFlow z perspektywy Klienta. Główne przypadki użycia obejmują utworzenie workflowu, jego uruchomienie, odczyt statusu oraz wyniku workflowu, a także anulowanie wykonania workflowu..



Rysunek 1 – Diagram przypadków użycia

PU-001: Utworzenie workflowu

Aktor: Klient

Cel: Utworzenie i zwalidowanie definicji workflowu zawierającego zadania i zależności między nimi.

Warunki wstępne: Klient ma autoryzowany dostęp do interfejsu API systemu.

Scenariusz:

1. Klient przesyła żądanie utworzenia workflowu wraz z jego definicją.
2. System weryfikuje poprawność struktury i topologię zależności.
3. System zapisuje definicję, nadaje jej unikalny identyfikator i ustawia stan początkowy.
4. System zwraca klientowi identyfikator utworzonego zasobu.

PU-002: Uruchomienie workflowu

Aktor: Klient

Cel: Rozpoczęcie asynchronicznego wykonywania zdefiniowanego workflowu.

Warunki wstępne: Workflow o podanym identyfikatorze istnieje w systemie.

Scenariusz :

1. Klient przesyła żądanie uruchomienia konkretnego workflowu.
2. System zmienia stan workflowu na aktywny
3. System identyfikuje pierwsze zadanie do wykonania i deleguje je do warstwy wykonawczej.
4. System zwraca klientowi potwierdzenie rozpoczęcia procesu.

PU-003: Pobranie stanu workflowu

Aktor: Klient

Cel: Uzyskanie informacji o aktualnym etapie przetwarzania workflowu

Warunki wstępne: Workflow został wcześniej utworzony w systemie.

Scenariusz:

1. Klient odpytuje system o stan, podając identyfikator workflowu.
2. System odczytuje główny stan procesu oraz statusy poszczególnych zadań.
3. System przygotowuje i zwraca klientowi skonsolidowany raport o przebiegu.

PU-004: Pobranie rezultatu zadania

Aktor: Klient

Cel: Pobranie danych wyjściowych wygenerowanych przez wykonanie zadania

Warunki wstępne: Wskazane zadanie wewnątrz przepływu ukończyło proces wykonywania.

Scenariusz:

1. Klient przesyła żądanie pobrania wyników dla wskazanego zadania.
2. System weryfikuje dostępność danych wynikowych.
3. System zwraca klientowi ładunek danych powiązany z wykonanym zadaniem.

PU-005: Anulowanie workflowu

Aktor: Klient

Cel: Natychmiastowe przerwanie przetwarzania wykonywanego workflowu.

Warunki wstępne: Workflow znajduje się w trakcie wykonywania.

Scenariusz:

1. Klient przesyła żądanie anulowania dla konkretnego workflowu.
2. System zmienia stan całego procesu na anulowany.
3. System powstrzymuje planowanie nowych zadań i wysyła sygnał przerwania dla zadań będących w toku.

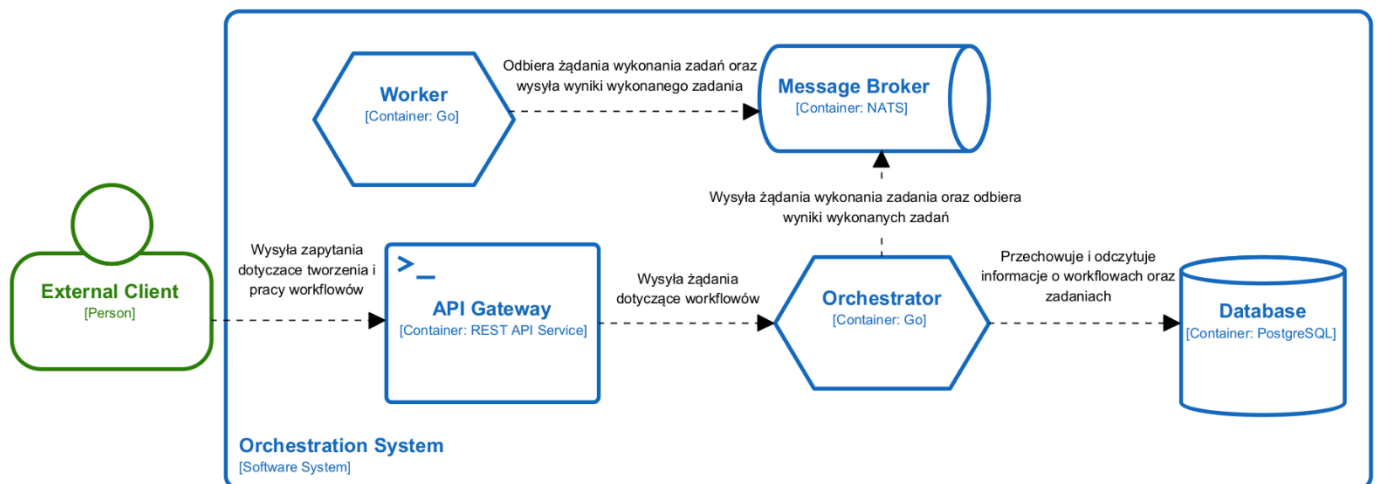
4.4. Architektura systemu

System TaskFlow został zaprojektowany jako zbiór współpracujących mikroservisów, których odpowiedzialności zostały rozdzielone zgodnie z realizowanymi funkcjami. Takie podejście pozwala oddzielić obsługę zapytań klienta, koordynowanie przebiegu wykonywania workflowu, wykonywania zadań oraz przechowywanie danych o stanie systemu.

Na poziomie architektury systemu można wyróżnić następujące główne elementy:

- API Gateway,
- Orkiestrator,
- Broker komunikatów NATS,
- Workerów,
- Bazę danych Postgres.

4.4.1. Komponenty i granice systemu



Rysunek 2 – Diagram Kontenerów

Klient komunikuje się z systemem za pośrednictwem API Gateway, który stanowi punkt wejścia systemu. Pozwala Klientowi wysyłać zapytania, uruchamiając dostępne funkcje takie jak, utworzenie workflowu, uruchomienie workflowu czy jego zatrzymanie.

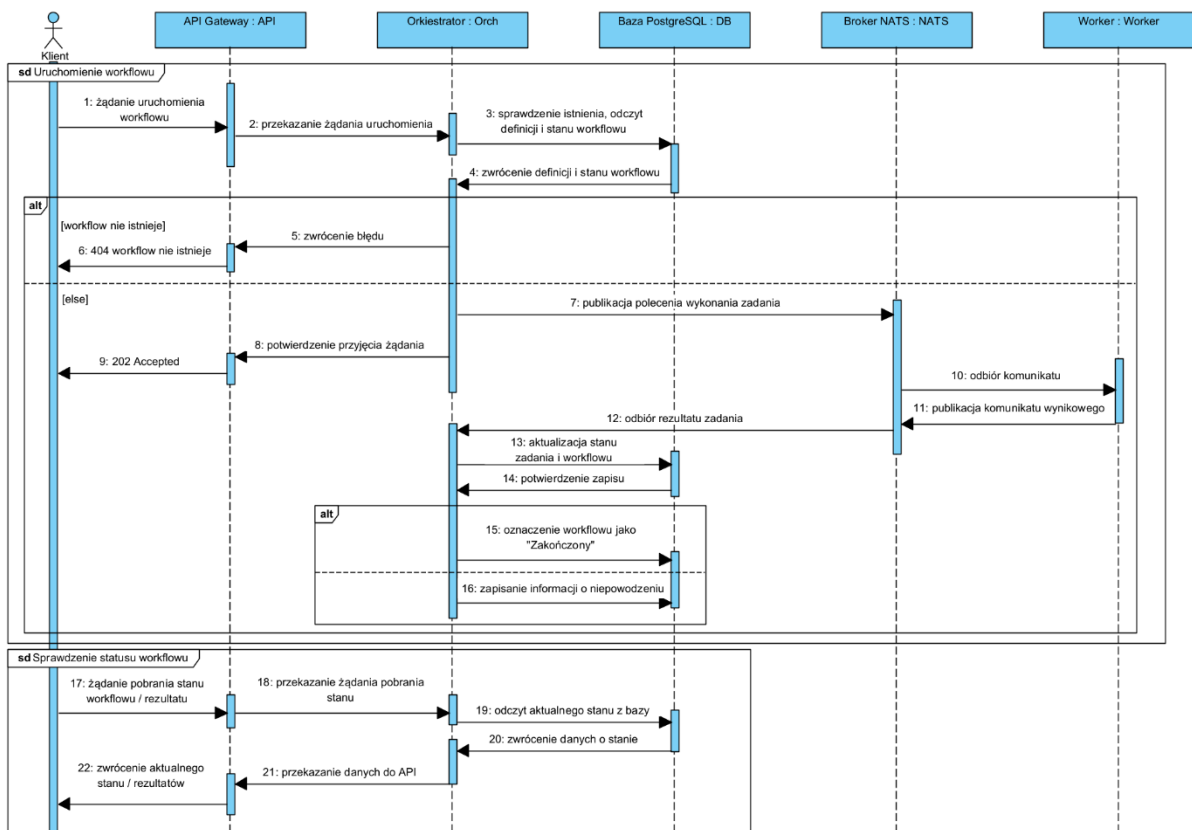
Orkiestrator to główny komponent systemu, zajmuje się zarządzaniem workflowami, nadzoruje wykonywanie zadań poprzez wysyłanie zapytań o wykonanie kolejnych zadań workflowami i na podstawie wyników decyduje czy należy kontynuować wykonywanie workflowu. Zapisuje on w bazie danych stan workflowu oraz poszczególnych zadań, oraz obsługuje zapytania utworzenia, wykonania, anulowania oraz zapytania o stan workflowu.

Baza danych PostgreSQL odpowiada za przechowywanie definicji workflowu oraz informacji związanych z ich wykonaniem. Przechowywany jest stan workflowu, zadania, zależności pomiędzy nimi i rezultaty ich wykonania. Stanowi źródło informacji dla Orkiestratora.

NATS, message broker pełni rolę pośrednika między Orkiestratorem, a Workerami. Subjecty NATS odbierają komunikaty na temat żądań wykonania zadania, jak i komunikaty z wynikami zakończonych zadań.

Workerzy to warstwa wykonawcza, odbierają zapytania wykonania zadań i je procesują. Dzięki rozdzieleniu od orkiestratora, pozwala na niezależne ich horyzontalne skalowanie, oznacza to że w razie jesteśmy w stanie zwiększyć liczbę instancji komponentu wykonawczego, a zadania mogą być rozdzielane pomiędzy większą liczbę workerów.

4.4.2. Przepływ wykonania workflowu



Rysunek 3 – Diagram Sekwencji

Współpraca komponentów systemu TaskFlow została przedstawiona na diagramie sekwencji. Diagram pokazuje dwa scenariusze, utworzenie workflowu i sprawdzenie jego statusu.

Wykonanie workflowu rozpoczyna się od przesłania przez klienta zapytania do API Gateway. Żądanie zawiera identyfikator workflowu przeznaczonego do uruchomienia. API Gateway przekazuje je do orkiestratora. Orkiestrator odczytuje w bazie danych definicje i aktualny stan danego workflowu. W przypadku gdy nie istnieje, zwraca błąd, który klient odbierze z kodem 404.

Po odczytaniu definicji orkiestrator analizuje zależności między zadaniami i określa, które z nich mogą zostać rozpoczęte. Zadania których wymagane zależności zostały spełnione, są przekazywane do wykonania za pośrednictwem brokera NATS, następnie zwraca do klienta informacje o zaakceptowaniu prośby o uruchomienie workflowu z kodem 202.

Instancja workera odbiera komunikat poprzez subskrypcję, a następnie wykonuje otrzymane zadanie. Po jego zakończeniu przygotowuje komunikat wynikowy. Komunikat zawiera informacje o powodzeniu lub niepowodzeniu operacji oraz dane związane z jej wykonaniem. Następnie worker publikuje rezultat do brokera NATS, skąd odbiera go orkiestrator.

Po otrzymaniu rezultatu orkiestrator aktualizuje stan zadania oraz workflowu w bazie PostgreSQL. Jeżeli zadanie zakończyło się pomyślnie, orkiestrator ponownie analizuje zależności i może przekazać do wykonania kolejne zadania. Jeżeli nie ma już zadań oczekujących, workflow zostaje oznaczony jako zakończony. W przypadku błędu orkiestrator zapisuje informacje o niepowodzeniu i stosuje przyjętą politykę obsługi błędów, obejmującą między innymi możliwość ponownego wykonania.

Przetwarzanie workflowu ma charakter asynchroniczny. Oznacza to, że API Gateway może zwrócić klientowi potwierdzenie przyjęcia żądania przed zakończeniem wszystkich zadań. Klient może później pobrać aktualny stan workflowu, wysyłając kolejne żądanie do API Gateway.

Na diagramie sekwencji pokazano również proces zapytania o stan workflowu.

4.4.3. Model danych i przechowywanie stanu

Bibliografia

- [1] Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018). *Fundamentals of business process management* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>
- [2] Manyika, J., Chui, M., Miremadi, M., Bughin, J., George, K., Willmott, P., & Dewhurst, M. (2017). *A future that works: Automation, employment, and productivity*. McKinsey Global Institute. <https://www.mckinsey.com/featured-insights/digital-disruption/harnessing-automation-for-a-future-that-works>
- [3] Busi, N., Gorrieri, R., Guidi, C., Lucchi, R., & Zavattaro, G. (2005). Choreography and orchestration: A synergic approach for system design. W: K. Barkaoui, A. Cavalcanti, & A. Cerone (red.), *Formal methods for open object-based distributed systems* (s. 228–240). Springer. https://doi.org/10.1007/11596141_18
- [4] Karagiannis, D. (1995). BPMS: Business process management systems. *ACM SIGOIS Bulletin*, 16(1), 10–13. <https://doi.org/10.1145/201977.201989>
- [5] Ouyang, C., van der Aalst, W. M. P., Dumas, M., ter Hofstede, A. H. M., & Rosa, M. (2007). Service-oriented processes: An introduction to BPEL. W: *Lecture Notes in Computer Science*. Springer. https://doi.org/10.1007/978-3-540-73522-9_1
- [6] Barbosa, D. M., Gadelha, R., Maia, P., Rocha, L., & Mendonça, N. (2018). Beethoven: An event-driven lightweight platform for microservice orchestration. *Proceedings of the 12th European Conference on Software Architecture Companion*, 1–4. <https://doi.org/10.1145/3241403.3241439>
- [7] Zapier. (n.d.). *How Zapier integrations work: Authentication, triggers, actions*. <https://docs.zapier.com/integrations/quickstart/how-zapier-works>
- [8] n8n. (n.d.). *n8n documentation*. <https://docs.n8n.io/>
- [9] Apache Airflow. (n.d.). *DAGs*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
- [10] Temporal. (n.d.). *Activity definition*. <https://docs.temporal.io/activity-definition>
- [11] Camunda. (n.d.). *Camunda 8 documentation*. <https://docs.camunda.io/>
- [12] Prefect. (n.d.). *Introduction*. <https://docs.prefect.io/v3/get-started>
- [13] Microsoft. (n.d.). *Durable orchestrations overview*. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/durable-task/common/durable-task-orchestrations>
- [14] NATS. (n.d.). *Core NATS*. NATS Documentation. <https://docs.nats.io/learn/core-nats/>

[15] Docker. (n.d.). *What is a container?* Docker Documentation. <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

[16] Kubernetes. (n.d.). *Overview*. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/overview/>